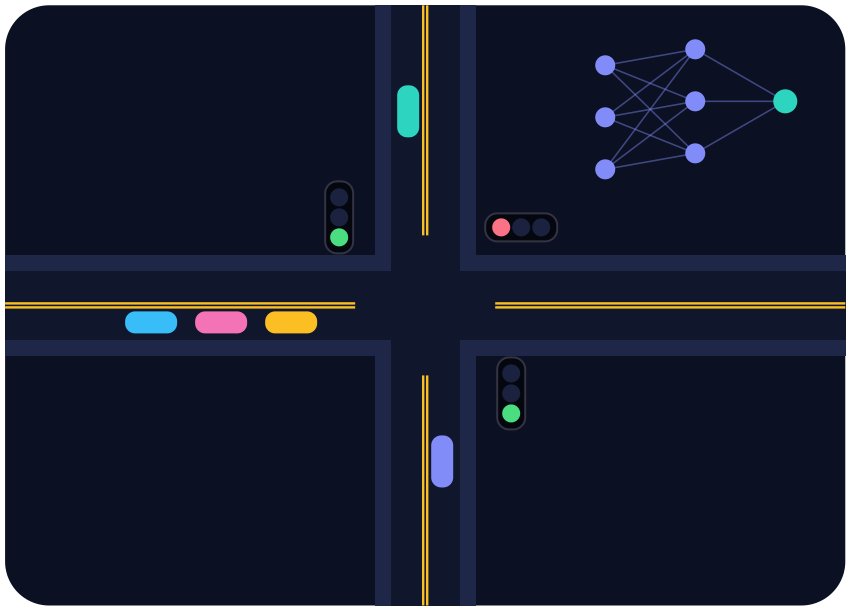
SEMÁFORO 2 · NOTAS DE TEORIA

Semáforo Neural

A teoria por trás do **semaforo2.html**: modelo de trânsito, medidas de desempenho, aprendizado por reforço, redes neurais e o algoritmo PPO, explicados do zero.



1. O problema do cruzamento
2. O modelo de trânsito
3. Como medir se está bom
4. Aprendizado por reforço
5. Redes neurais
6. O algoritmo PPO
7. Os outros controladores
8. Resultados e limitações
9. Glossário e referências

Documento de apoio ao simulador **semaforo2.html** (projeto pendulo_html). Os números citados são os parâmetros usados no simulador e resultados de testes feitos com ele.

1 O problema do cruzamento

O simulador mostra um cruzamento de duas ruas de mão dupla, com uma faixa em cada sentido. Os carros só seguem em frente (não há conversões). Há duas **fases**: na fase **N–S** o verde é para quem vem do norte e do sul; na fase **L–O**, para quem vem do leste e do oeste. As duas nunca ficam verdes ao mesmo tempo.

Trocar de fase não é instantâneo. Quando o controlador decide trocar, o verde atual vira **amarelo por 3 s**, depois todos ficam **vermelhos por 2 s** (para esvaziar o miolo do cruzamento) e só então a outra via recebe o verde. Cada troca, portanto, "custa" pelo menos 5 s em que ninguém anda, e ainda os primeiros carros de cada fila demoram para arrancar. Para evitar trocas absurdas, todo verde dura no mínimo **5 s** e no máximo **90 s**.

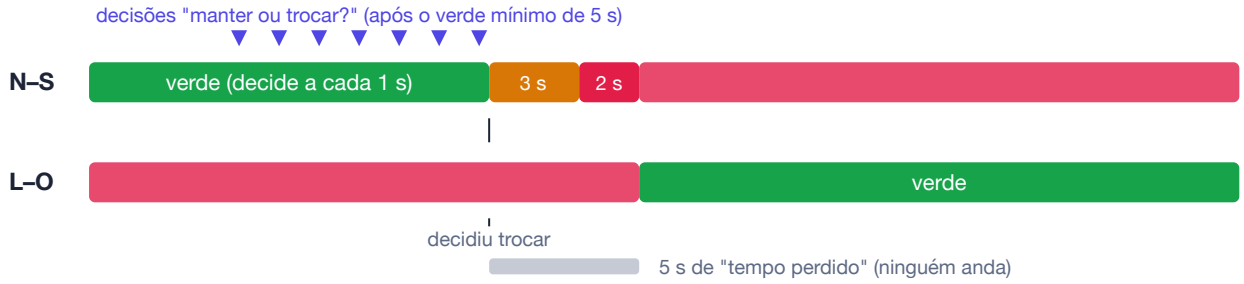


Figura 1. O ciclo do semáforo. O controlador só escolhe *quando* trocar; amarelo e vermelho geral são automáticos.

O dilema

Todo semáforo vive entre dois erros. Se trocar **pouco**, a via no vermelho acumula uma fila enorme e espera muito, mesmo quando a via verde já está vazia. Se trocar **demais**, o cruzamento passa boa parte do tempo em amarelo e vermelho geral, a capacidade cai e as filas crescem nas duas vias. Existe um ponto ótimo, e ele muda o tempo todo, conforme o volume de carros de cada via.

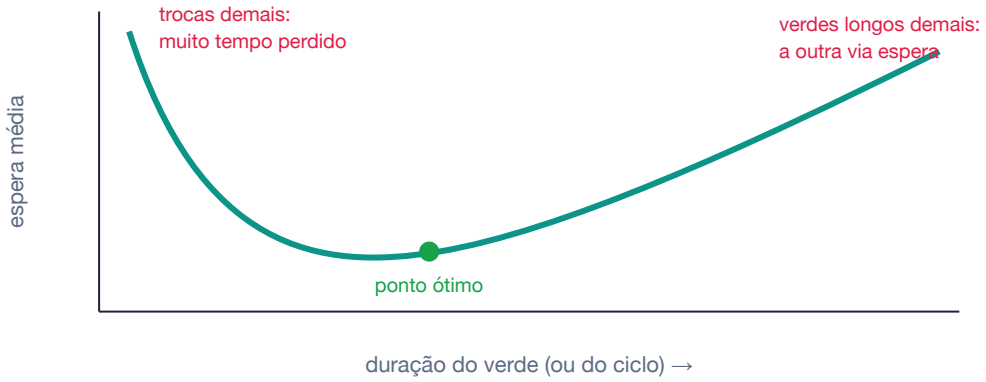


Figura 2. A curva em "U" da espera em função do tempo de verde, conhecida desde os estudos de Webster (1958).

O plano mais antigo e comum é o de **tempo fixo**: cada via recebe sempre o mesmo verde (no simulador, 30 s), haja carro ou não. Ele é simples, mas não reage ao trânsito. Os controladores do semaforo2 tentam fazer melhor decidindo, segundo a segundo, se vale a pena manter o verde.

2 O modelo de trânsito

2.1 Chegada dos carros: processo de Poisson

Em uma rua real, os carros não chegam em intervalos regulares. O modelo clássico para chegadas independentes é o **processo de Poisson**: em um intervalo curtíssimo *dt*, a chance de chegar um carro é proporcional a ele.

$$P(\text{chegar um carro em } dt) = \lambda(t) \cdot dt$$

A taxa λ (carros por segundo) vem dos sliders de fluxo. Para imitar a **hora do rush**, ela oscila em uma senoide de 30 minutos, e as duas ruas ficam em oposição: quando uma enche, a outra esvazia.

$$\lambda_{\text{N–S}}(t) = \frac{f_{\text{N–S}}}{60}\left(1 + A\sin\frac{2\pi t}{1800}\right) \qquad \lambda_{\text{L–O}}(t) = \frac{f_{\text{L–O}}}{60}\left(1 - A\sin\frac{2\pi t}{1800}\right)$$

Aqui *f* é o fluxo em carros por minuto por sentido e *A* é a "variação" (50% por padrão). Um controlador de tempo fixo, ajustado para a média, fica errado na maior parte do tempo; um controlador que aprende pode se adaptar.

2.2 O motorista: Intelligent Driver Model (IDM)

Cada carro é dirigido pelo **IDM** (Treiber, Hennecke e Helbing, 2000), um dos modelos de "seguir o carro da frente" mais usados em pesquisa de tráfego. Ele dá a aceleração de um carro a partir da sua velocidade *v*, da distância livre *s* até o carro da frente e da diferença de velocidade Δv entre os dois:

$$\frac{dv}{dt} = a\left[1 - \left(\frac{v}{v_0}\right)^4 - \left(\frac{s^*(v,\Delta v)}{s}\right)^2\right] \qquad s^* = s_0 + vT + \frac{v\Delta v}{2\sqrt{ab}}$$

O termo $(v/v_0)^4$ faz o carro acelerar até a velocidade desejada v_0. O termo $(s^*/s)^2$ é o freio: s^* é a distância que o motorista *quer* manter (uma folga mínima, mais o tempo de reação vezes a velocidade, mais uma parcela que cresce quando ele se aproxima rápido do carro da frente). Quando a distância real *s* fica menor que a desejada, ele freia; quanto menor, mais forte.

Parâmetro	Significado	Valor
v_0	velocidade desejada	13,9 m/s (50 km/h)
a	aceleração máxima confortável	1,5 m/s²
b	desaceleração confortável	2,0 m/s²
T	tempo de distância (headway) desejado	1,2 s
s_0	distância mínima parado	2 m
—	comprimento do carro	4,5 m

2.3 Semáforo vermelho e o dilema do amarelo

Para o IDM, um sinal vermelho é simplesmente um "carro parado" invisível sobre a faixa de retenção: o motorista freia até parar antes dela, exatamente como faria atrás de outro carro. O amarelo cria o famoso **dilema do amarelo**: frear ou seguir? O simulador usa a regra física: o motorista consegue parar com uma frenagem de até $b_{\max}$ = 3,5 m/s² se a distância até a faixa for maior que a distância de frenagem.

$$\text{segue se } d < \frac{v^2}{2b_{\max}}, \quad \text{senão para}$$

Quem decide seguir fica "comprometido" e atravessa mesmo que o sinal fique vermelho logo depois, e é para isso que servem os 2 s de vermelho geral.

> **FENÔMENO QUE APARECE SOZINHO**
>
> Ninguém programou "fluxo de saturação" nem "tempo perdido no arranque": eles surgem do IDM. Uma fila que recebe o verde descarrega cerca de um carro a cada 1,7 a 2 s, e os primeiros demoram mais porque partem do repouso. É por isso que trocar de fase demais é caro.

3 Como medir se está bom

3.1 Atraso (espera)

Um carro que surge 150 m antes do cruzamento e encontra tudo livre chega à faixa em t_{ideal} = 150 m ÷ 13,9 m/s ≈ **10,8 s**. O **atraso** de cada carro é quanto ele demorou a mais que isso. O gráfico "espera" mostra a média desse atraso a cada 2 minutos simulados.

$$\text{atraso} = t_{\text{real}} - t_{\text{ideal}}$$

3.2 Índice de eficiência

Para ter um número que **sobe** quando melhora, o simulador usa a razão entre o tempo ideal e o tempo real (o inverso do *Travel Time Index* usado em engenharia de tráfego):

$$\eta = \frac{t_{\text{ideal}}}{t_{\text{ideal}} + \text{atraso médio}}$$

η = 100% significa que ninguém parou; com 11 s de atraso médio, η ≈ 50%. A **vazão** (carros por minuto que cruzam a faixa) também é mostrada, mas ela engana: enquanto o cruzamento dá conta do trânsito, todo carro que chega acaba passando, e a vazão fica igual à chegada, seja o controle bom ou ruim. Ela só diferencia os controladores quando o trânsito satura.

3.3 Uma comparação justa: o cruzamento de referência

Como as chegadas são aleatórias, comparar "hoje" com "ontem" mistura sorte e mérito. O simulador resolve isso com um **cruzamento-sombra**, invisível, controlado pelo tempo fixo e que recebe **exatamente os mesmos carros, nos mesmos instantes**. A linha tracejada do gráfico e o selo "tempo fixo" vêm dele. É a mesma ideia de um experimento controlado: tudo igual, exceto o controlador.

3.4 Por que punir "carros parados"

A **lei de Little** (1961) liga três grandezas de qualquer fila estável: número médio de clientes na fila = taxa de chegada × tempo médio de espera. Como a taxa de chegada não depende do semáforo, **minimizar o número de carros parados, somado ao longo do tempo, é o mesmo que minimizar a espera total**. Por isso a recompensa do aprendizado é simplesmente "menos carros parados".

$$\sum_{\text{carros}} \text{espera} = \int N(t)\,dt \quad (N(t) = \text{carros parados no instante } t)$$

4 Aprendizado por reforço

Aprendizado por reforço (RL, do inglês *reinforcement learning*) é aprender por tentativa e erro: um **agente** observa o **estado** do ambiente, escolhe uma **ação** e recebe uma **recompensa**. Ninguém diz qual era a ação certa; o agente só descobre, com o tempo, quais escolhas levam a recompensas maiores.

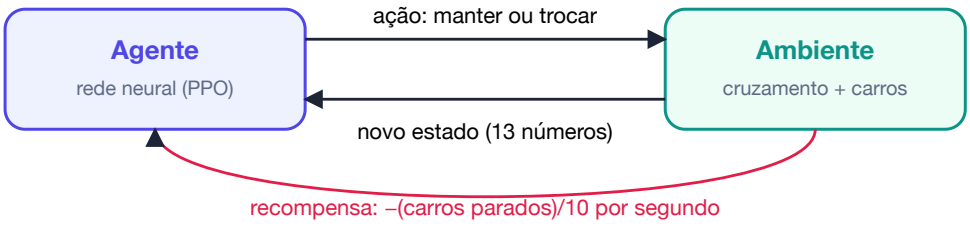


Figura 3. O ciclo do aprendizado por reforço aplicado ao semáforo.

4.1 Estado, ação e recompensa no semáforo

- **Estado** *s*: o que o agente vê. No PPO são 13 números, sempre do ponto de vista da fase que está verde (as duas vias verdes primeiro, depois as duas vermelhas). Para cada via: carros parados (÷10), carros presentes nos 100 m antes da faixa (÷10) e a espera do carro mais antigo (÷60 s). O 13º número é há quanto tempo o verde está aceso (÷60 s). Dividir os valores deixa tudo na faixa de 0 a 1, o que facilita o aprendizado da rede.
- **Ação** *a*: manter o verde ou trocar. É escolhida uma vez por segundo, depois do verde mínimo.
- **Recompensa** *r*: −(carros parados)/10 a cada segundo. Pela lei de Little, maximizar a soma das recompensas é minimizar a espera total.

4.2 Retorno e desconto

Uma decisão afeta o futuro: segurar o verde agora pode ser bom para esta via e ruim para a outra daqui a 20 s. O agente busca maximizar o **retorno**, a soma das recompensas futuras, com as mais distantes valendo menos:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots \qquad \gamma = 0{,}97 \text{ por segundo}$$

O fator de desconto γ define o "horizonte" do agente: com 0,97 por segundo, ele se importa com algo como $1/(1-\gamma) \approx 33$ s à frente, o suficiente para enxergar um ciclo de semáforo.

4.3 Decisões em intervalos desiguais (semi-Markov)

No semáforo, as decisões não são igualmente espaçadas: "manter" leva à próxima decisão em 1 s, mas "trocar" só permite decidir de novo depois de amarelo + vermelho geral + verde mínimo, 10 s depois. Esse tipo de problema é um **processo de decisão semi-Markoviano**. A solução é descontar pelo tempo decorrido: a recompensa acumulada durante o intervalo Δt entra com o desconto de cada segundo, e o valor do estado seguinte é multiplicado por $\gamma^{\Delta t}$.

4.4 Função valor e erro de diferença temporal (TD)

A **função valor** *V(s)* estima o retorno esperado a partir do estado *s*: "quão boa é esta situação?". Ela é aprendida comparando a previsão com o que aconteceu de fato um passo depois. A diferença é o **erro TD**:

$$\delta = \underbrace{R + \gamma^{\Delta t} V(s')}_{\text{o que aconteceu + previsão do resto}} - \underbrace{V(s)}_{\text{previsão antiga}}$$

Se $\delta > 0$, as coisas saíram melhores que o esperado; se $\delta < 0$, piores. Esse "sinal de surpresa" é a matéria-prima de todo o aprendizado.

4.5 Política estocástica e exploração

A **política** π é a regra de decisão. No PPO ela é **estocástica**: a rede calcula um número z e a probabilidade de trocar é dada pela função logística (sigmoide):

$$\pi(\text{trocar} \mid s) = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Sortear a ação, em vez de sempre escolher a mais provável, é o que permite **explorar**: o agente às vezes testa algo diferente e descobre se era melhor. No início $z \approx 0$ e a chance é 50%; com o tempo, as probabilidades se aproximam de 0% ou 100% nas situações em que o agente tem certeza.

4.6 Ator-crítico e gradiente de política

Os métodos **ator-crítico** usam duas peças: o **ator** (a política, que decide) e o **crítico** (a função valor, que julga). A ideia central é o **gradiente de política**: aumentar a probabilidade das ações que deram resultado melhor que o esperado e diminuir a das que deram pior. "Melhor que o esperado" é medido pela **vantagem** *A*, que o crítico ajuda a calcular:

$$\nabla_\theta J = \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, A(s,a) \right]$$

Aqui θ são os pesos da rede do ator. O termo $\nabla \log \pi$ aponta "para onde mexer os pesos para tornar esta ação mais provável"; multiplicado pela vantagem, ele empurra a política na direção certa.

5 Redes neurais

No semáforo 1, o ator e o crítico eram tabelas de 75 regras fuzzy. No semáforo 2, cada um é uma **rede neural**, que consegue representar relações muito mais ricas entre os 13 números do estado.

5.1 O neurônio e as camadas

Um neurônio artificial faz uma soma ponderada das entradas, soma um viés e passa o resultado por uma função não linear. O simulador usa a tangente hiperbólica (tanh), que vai de −1 a 1:

$$h = \tanh\left(\sum_i w_i x_i + b \right)$$

Neurônios em camadas formam um **perceptron multicamadas** (MLP). O ator e o crítico têm a mesma forma: 13 entradas → 32 neurônios → 32 neurônios → 1 saída. Para o ator, a saída é o *z* que vira proba-

bilidade de trocar; para o crítico, é o valor *V(s)*. Cada rede tem 13·32 + 32 + 32·32 + 32 + 32 + 1 = **1.537 pesos**.

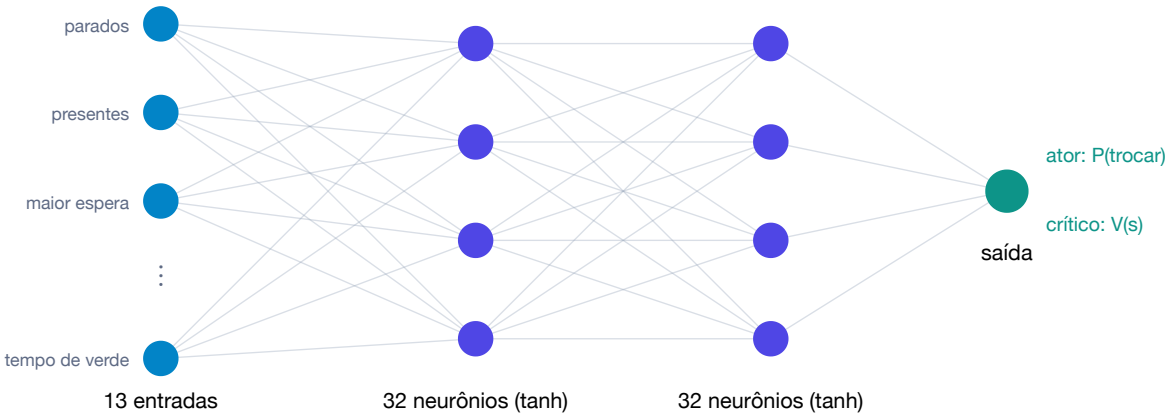


Figura 4. A arquitetura das redes do ator e do crítico (só alguns neurônios desenhados).

5.2 Como a rede aprende: gradiente e retropropagação

Aprender é ajustar os pesos para diminuir uma **perda** (um número que mede o erro). O **gradiente** diz, para cada peso, em que direção mexer para a perda cair. Calcular o gradiente de milhares de pesos de uma vez é o papel da **retropropagação** (*backpropagation*): aplica-se a regra da cadeia do cálculo, da saída para a entrada, camada por camada. Para a tanh, a derivada usada é 1 − tanh².

5.3 O otimizador Adam

Em vez de dar sempre o mesmo passo na direção do gradiente, o **Adam** (Kingma e Ba, 2015) mantém duas médias móveis para cada peso: a do gradiente (que dá "inércia" à direção) e a do gradiente ao quadrado (que mede a escala). O passo de cada peso é a primeira dividida pela raiz da segunda, então pesos com gradientes ruidosos andam devagar e pesos com gradientes consistentes andam mais rápido.

$$m_t = 0{,}9\,m_{t-1} + 0{,}1\,g, \quad v_t = 0{,}999\,v_{t-1} + 0{,}001\,g^2, \quad w \leftarrow w - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

No simulador, a taxa de aprendizado α é 0,001 para o ator e 0,002 para o crítico, multiplicadas pelo slider "Taxa de aprendizado". Além disso, o tamanho total do gradiente é limitado (**recorte da norma do gradiente**), uma proteção contra passos gigantes causados por um lote de dados atípico.

5.4 Inicialização

Os pesos começam aleatórios, com escala proporcional a 1/√(nº de entradas), para que os sinais não explodam nem sumam ao atravessar as camadas. A última camada do ator começa muito pequena (×0,01): assim a rede nasce "indecisa", com 50% de chance de trocar em qualquer situação, e não com um viés arbitrário.

6 O algoritmo PPO

O **PPO** (*Proximal Policy Optimization*, Schulman et al., 2017) é um método ator-crítico que se tornou um dos padrões do aprendizado por reforço profundo, usado de robótica a jogos e no ajuste fino de modelos de linguagem. Ele resolve o principal problema do gradiente de política com redes neurais: **um passo grande demais pode estragar a política**, e uma política ruim gera dados ruins, que pioram ainda mais o aprendizado.

6.1 A razão de probabilidades

O PPO coleta experiência com a política atual ("antiga") e depois treina a rede várias vezes sobre esses mesmos dados. Para saber o quanto a nova política se afastou da antiga, ele mede a razão:

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{antiga}}(a_t \mid s_t)}$$

$\rho = 1$ significa "a mesma probabilidade de antes"; $\rho = 1{,}5$, "50% mais provável agora".

6.2 O objetivo recortado (clipping)

O coração do PPO é este objetivo, que o ator tenta maximizar:

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\right)\right], \quad \varepsilon = 0{,}2$$

A leitura é intuitiva. Se uma ação foi **melhor** que o esperado ($\hat{A} > 0$), vale torná-la mais provável, mas só até 20% a mais ($\rho = 1{,}2$); dali em diante o ganho é cortado e o gradiente zera. Se foi **pior** ($\hat{A} < 0$), vale torná-la menos provável, mas só até 20% a menos. O "min" garante que o objetivo seja uma estimativa pessimista: nunca se ganha nada por se afastar demais da política antiga.

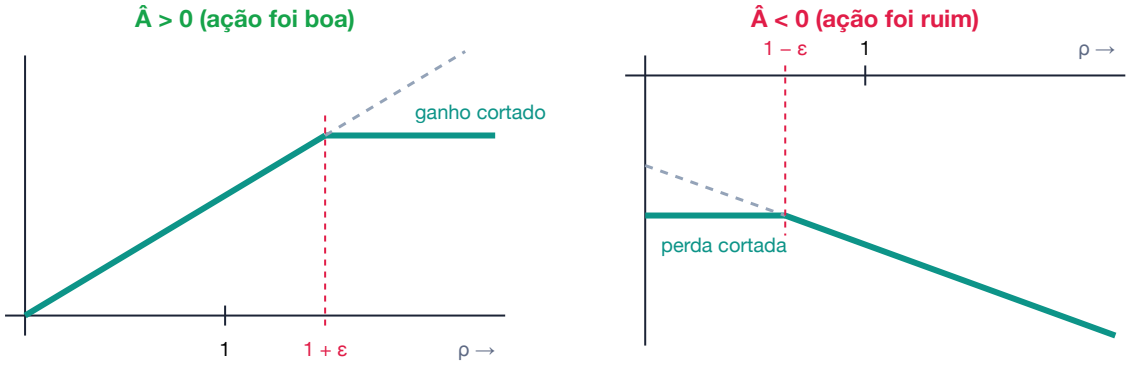


Figura 5. O objetivo do PPO (em verde-água) em função da razão ρ. Além da faixa ±ε, o objetivo fica plano e a rede para de empurrar a política naquela direção.

6.3 Vantagem com GAE

Para estimar a vantagem $\hat{A}$, o simulador usa a **GAE** (*Generalized Advantage Estimation*, Schulman et al., 2016), uma média ponderada de erros TD futuros:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\delta_{t+1} + (\gamma\lambda)^2\delta_{t+2} + \cdots, \quad \lambda = 0{,}9$$

O parâmetro λ regula um compromisso. Com $\lambda = 0$, só o próximo passo conta (estimativa estável, mas que depende muito de um crítico ainda impreciso). Com $\lambda = 1$, conta o retorno inteiro observado (sem viés, mas muito ruidoso). Com intervalos desiguais, cada termo usa $\gamma^{\Delta t}$ no lugar de γ. Antes do treino, as vantagens de cada lote são **normalizadas** (média 0, desvio 1), o que deixa o tamanho dos passos independente da escala das recompensas.

6.4 Crítico e entropia

O crítico é treinado para prever o retorno observado, minimizando o erro quadrático ($V(s)$ − alvo)², em que o alvo é a vantagem somada ao valor previsto na coleta ($\hat{A} + V_{\text{coleta}}$). O ator recebe ainda um pequeno **bônus de entropia**: a entropia mede o quanto a política ainda está "em dúvida" e, ao recompensá-la um pouco, evita que a rede fique certa cedo demais e pare de explorar.

$$H = -p\log p - (1-p)\log(1-p), \qquad \text{coeficiente} = 0{,}01 \times \text{exploração}$$

6.5 O ciclo de treino

1. **Coletar:** o agente dirige os cruzamentos com a política atual até juntar 256 decisões, guardando estado, ação, probabilidade, valor previsto e recompensa recebida.
2. **Calcular vantagens:** GAE de trás para frente em cada trajetória.
3. **Treinar:** 6 passadas (épocas) sobre os dados, em lotes de 64 decisões embaralhadas, ajustando ator e crítico com Adam.
4. **Descartar e repetir:** os dados são jogados fora e uma nova coleta começa com a política melhorada. O PPO é *on-policy*: só aprende com dados da sua própria política recente.

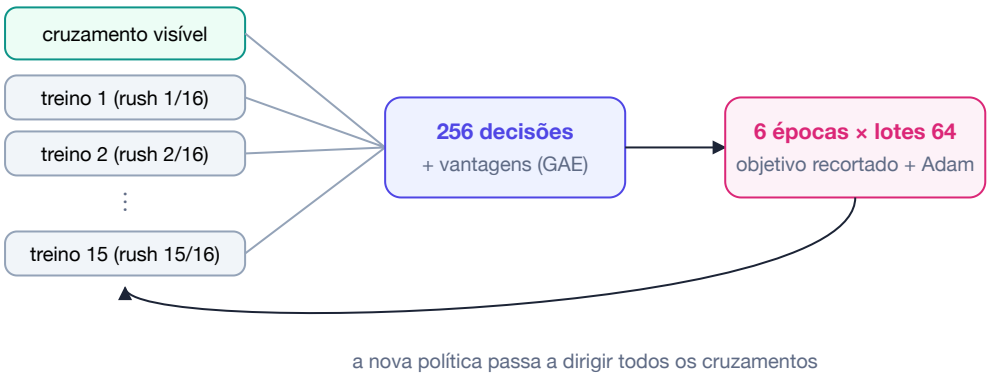


Figura 6. Atores paralelos alimentam um único lote de experiência; a rede treinada volta a dirigir todos eles.

6.6 Atores paralelos

O PPO precisa de muita experiência, e um único cruzamento gera só uma decisão por segundo. A solução clássica (popularizada pelo A3C, Mnih et al., 2016, e usada no próprio artigo do PPO) é rodar **vários ambientes em paralelo** com a mesma rede. O semaforo2 simula até **15 cruzamentos invisíveis** com os mesmos fluxos, mas cada um em um momento diferente da hora do rush. Isso traz duas vantagens: muito mais dados por minuto simulado e dados mais variados (um lote mistura situações de rush e de calmaria, o que evita que a rede "esqueça" uma delas). Um cruzamento de treino que trava por completo (mais de 150 carros parados) é reiniciado, para continuar gerando situações úteis. O slider "Cruzamentos de treino" permite ver o efeito: com 0, a rede aprende só com o cruzamento visível e demora muito mais.

6.7 Decisões que o agente não tomou

Às vezes a troca acontece sem que a rede tenha escolhido: quando o verde atinge 90 s ou quando o usuário clica no cruzamento. Essas decisões entram no cálculo do crítico (as recompensas continuam valendo), mas são excluídas do objetivo do ator, que só aprende com escolhas que ele próprio sorteou.

> **RESUMO DOS HIPERPARÂMETROS DO PPO**
> γ = 0,97/s · λ (GAE) = 0,9 · ε (clip) = 0,2 · 256 decisões por atualização · 6 épocas · lotes de 64 · Adam com α = 0,001 (ator) e 0,002 (crítico) · bônus de entropia 0,01 · redes 13–32–32–1 com tanh · até 15 atores paralelos.

7 Os outros controladores

7.1 Tempo fixo

Cada fase recebe sempre o mesmo verde (slider "Verde do tempo fixo", 30 s por padrão). É a referência de comparação e o que roda no cruzamento-sombra.

7.2 Lógica fuzzy: conjuntos, pertinência e regras

Na lógica fuzzy (Zadeh, 1965), uma afirmação pode ser "um pouco verdadeira". A **pertinência** μ, entre 0 e 1, diz o quanto um valor pertence a um conjunto. Com conjuntos triangulares, 5 carros na fila podem ser 60% "fila média" e 40% "fila pequena". O simulador usa 5 conjuntos para os carros na via verde, 5 para os carros na via vermelha (centros em 0, 3, 7, 12 e 20 carros) e 3 para o tempo de verde (5, 20 e 45 s): 5 × 5 × 3 = **75 regras**.

No modelo de **Takagi-Sugeno** (1985), cada regra conclui um número (uma "urgência de trocar" *w*), e a decisão é a média dessas conclusões ponderada pelo quanto cada regra se aplica:

$$u = \sum_i \varphi_i(s)\, w_i, \quad \varphi_i = \mu_{\text{verde}} \cdot \mu_{\text{vermelho}} \cdot \mu_{\text{tempo}}, \quad \text{troca se } u > 0$$

7.3 Especialista

As 75 urgências vêm de uma fórmula de engenharia de tráfego, aplicada ao centro de cada regra: atender a fila maior, não cortar um verde que ainda flui e ficar mais propenso a trocar quando o verde se alonga.

$$w_i = 0{,}5 \cdot \text{carros}_{\text{vermelho}} - 0{,}8 \cdot \text{carros}_{\text{verde}} - 0{,}8 + 0{,}04 \cdot (t_{\text{verde}} - 5)$$

7.4 Fuzzy com ator-crítico (o semáforo 1)

Mesmas 75 regras, mas as urgências começam em zero e são aprendidas. O ator e o crítico são **lineares nas pertinências**: cada regra tem um peso de ação e um de valor. A exploração é feita somando ruído gaussiano à urgência, e o aprendizado usa o erro TD com **traços de elegibilidade** (λ = 0,8), que distribuem o crédito para as regras usadas nas decisões recentes (Barto, Sutton e Anderson, 1983).

	Fuzzy ator-crítico	PPO
O que vê	3 números: carros no verde, no vermelho, tempo de verde	13 números: parados, presentes e maior espera de cada via, tempo de verde
Representação	75 regras (linear nas pertinências)	2 redes neurais de 1.537 pesos
Atualização	a cada decisão (online)	em lotes de 256, várias épocas
Dados	só o cruzamento visível	visível + até 15 paralelos
Estabilidade do passo	taxa fixa	objetivo recortado + Adam + normalização
Interpretação	fácil: cada regra tem significado	difícil: "caixa-preta" (o mapa da política ajuda)

8 Resultados e limitações

Testes do simulador com 4 execuções de 4 horas simuladas por caso (eficiência média da última hora; quanto maior, melhor):

Trânsito	PPO	Fuzzy AC	Especialista	Tempo fixo (espera)
Normal (9 e 5 carros/min, ±50%)	**48–50%**	47–48%	45–49%	26–42 s
Pesado (11 e 7 carros/min, ±50%)	**34–39%**	**5–40%**	29–33%	52–109 s

- No trânsito normal, todos os controladores adaptativos ficam próximos: o problema é simples e já há pouco a ganhar. Todos reduzem a espera do tempo fixo pela metade ou mais.
- No trânsito pesado, a diferença aparece: o PPO é estável e supera até as regras do especialista, enquanto o fuzzy falhou em 2 de 4 execuções. A informação extra (espera do carro mais antigo de cada via) e o treino mais estável fazem diferença justamente quando o sistema está perto do limite.
- O PPO converge em cerca de 20 a 30 minutos simulados com 15 atores paralelos. No início, como qualquer aprendiz, ele deixa as filas crescerem.

Limitações do modelo

- Um único cruzamento isolado, uma faixa por sentido, sem conversões nem pedestres. Em cidades reais, os semáforos interagem ("onda verde"), um tema de pesquisa atual (RL multiagente).
- A rede é treinada em simulação. Levar para a rua exige lidar com sensores imperfeitos e com a diferença entre simulação e realidade (*sim-to-real*).
- O PPO não dá garantias formais de estabilidade nem de desempenho mínimo, ao contrário do controle clássico. Na prática, sistemas reais combinam aprendizado com regras de segurança (verdes mínimo e máximo, como aqui).
- O mapa da política é uma aproximação: ele mostra a rede em situações sintéticas (carros divididos igualmente entre as duas vias de cada fase), não todas as combinações possíveis.

9 Glossário e referências

Agente	quem decide (aqui, o controlador do semáforo).
Ator / crítico	a parte que escolhe ações / a parte que estima o valor das situações.
Desconto (γ)	quanto uma recompensa futura vale menos que uma imediata.
Entropia	medida de incerteza da política; alta = ainda explorando.
Época	uma passada completa de treino sobre o lote de dados.
Erro TD (δ)	diferença entre o que aconteceu e o que o crítico previa.
GAE	forma de estimar a vantagem combinando vários erros TD futuros.
On-policy	aprende só com dados gerados pela própria política atual.
Política (π)	a regra de decisão: probabilidade de cada ação em cada estado.
Vantagem (A)	quanto uma ação foi melhor ou pior que a média esperada naquele estado.

1. R. S. Sutton e A. G. Barto. *Reinforcement Learning: An Introduction*, 2ª ed. MIT Press, 2018.
2. J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov. "Proximal Policy Optimization Algorithms". arXiv:1707.06347, 2017.
3. J. Schulman, P. Moritz, S. Levine, M. Jordan e P. Abbeel. "High-Dimensional Continuous Control Using Generalized Advantage Estimation". ICLR, 2016.
4. D. P. Kingma e J. Ba. "Adam: A Method for Stochastic Optimization". ICLR, 2015.
5. V. Mnih et al. "Asynchronous Methods for Deep Reinforcement Learning" (A3C). ICML, 2016.
6. M. Treiber, A. Hennecke e D. Helbing. "Congested traffic states in empirical observations and microscopic simulations" (IDM). *Physical Review E* 62, 1805, 2000.
7. F. V. Webster. "Traffic Signal Settings". Road Research Technical Paper 39, HMSO, Londres, 1958.
8. J. D. C. Little. "A Proof for the Queuing Formula L = λW". *Operations Research* 9(3), 1961.
9. T. Takagi e M. Sugeno. "Fuzzy identification of systems and its applications to modeling and control". *IEEE Trans. SMC* 15(1), 1985.
10. A. G. Barto, R. S. Sutton e C. W. Anderson. "Neuronlike adaptive elements that can solve difficult learning control problems". *IEEE Trans. SMC* 13(5), 1983.
11. H. Wei, G. Zheng, V. Gayah e Z. Li. "A Survey on Traffic Signal Control Methods". arXiv:1904.08117, 2019.